University of Central Missouri

Department of Computer Science & Cybersecurity

CS5760 Natural Language Processing

Spring 2026

Homework 4.

Student name:

Submission Requirements:

- Once finished your assignment push your source code to your repo (GitHub) and explain the work through the ReadMe file properly. Make sure you add your student info in the ReadMe file.
- Submit your GitHub link on the Bright Space.
- Comment your code appropriately ***IMPORTANT.***
- Any submission after provided deadline is considered as a late submission.

Q6. Multi-Input Feedforward

Consider a neural network with 3 inputs x1=2, x2=1, x3=3, one hidden layer of 2 sigmoid units, and one sigmoid output.
Weights:

$$A = \begin{bmatrix} 0.2 & -0.5 \\ 0.1 & 0.3 \\ -0.2 & 0.8 \\ 0.4 & -0.6 \end{bmatrix}, \quad B = \begin{bmatrix} 0.7 \\ -1.2 \\ 0.5 \end{bmatrix}$$

- A includes biases (last row).
- B includes bias (last element).

Tasks
a) Compute hidden pre-activations and activations.
b) Compute the output activation y.
c) If the true label is t=1, calculate the binary cross-entropy loss.

Solution:

Given: $x_1 = 2,\ x_2 = 1,\ x_3 = 3$
Add Bias: $x = [2,1,3,1]$
Compute $z_1 = (2)(0.2) + (1)(0.1) + (3)(-0.2) + (1)(0.4)$
$= 0.4 + 0.1 - 0.6 + 0.4 = 0.3$
Compute $z_2 = (2)(-0.5) + (1)(0.3) + (3)(0.8) + (1)(-0.6)$
$= -1 + 0.3 + 2.4 - 0.6 = 1.1$
Apply Sigmoid

$$h_1 = \sigma(0.3) = \frac{1}{1 + e^{-0.3}}$$
$$e^{-0.3} \approx 0.741$$
$$h_1 \approx \frac{1}{1.741} \approx 0.574$$

$$h_2 = \sigma(1.1) = \frac{1}{1 + e^{-1.1}}$$
$$e^{-1.1} \approx 0.333$$
$$h_2 \approx \frac{1}{1.333} \approx 0.750$$

b.

Solution: Add bias: $h = [0.574,\ 0.750,\ 1]$
Compute output $z = (0.574)(0.7) + (0.750)(-1.2) + (1)(0.5)$

$$z = 0.402 - 0.900 + 0.5 = 0.002$$

Final Output (Sigmoid): $y = \sigma(0.002)$

$$e^{-0.002} \approx 0.998$$
$$y \approx \frac{1}{1.998} \approx 0.5005$$

C. Solution: Loss (Binary Cross-Entropy): $t = 1$

$$L = -\log(y)$$
$$L = -\log(0.5005)$$
$$L \approx 0.692$$

Q7. XOR with ReLU Network

We use the XOR ReLU network from the slides

$$h_1 = \mathrm{ReLU}(x_1 + x_2), \quad h_2 = \mathrm{ReLU}(x_1 + x_2 - 1), \quad y = \mathrm{ReLU}(h_1 - 2h_2)$$

Extend it with an additional hidden unit:

$$h_3 = \mathrm{ReLU}(2x_1 - x_2)$$

and new output:

$$y = \mathrm{ReLU}(h_1 - 2h_2 + h_3)$$

Tasks

a) Compute outputs for all four XOR inputs (0,0),(0,1),(1,0),(1,1).
b) Compare the decision boundary with the original 2-hidden-unit XOR network.
c) Does this extension still compute XOR exactly? If not, which inputs differ?

Solution:

Case 1: (0, 0)

$$h_1 = \mathrm{ReLU}(0 + 0) = 0$$
$$h_2 = \mathrm{ReLU}(0 + 0 - 1) = \mathrm{ReLU}(-1) = 0$$
$$h_3 = \mathrm{ReLU}(0 - 0) = 0$$
$$y = \mathrm{ReLU}(0 - 0 + 0) = 0$$

Case 2: (0, 1)

$$h_1 = \mathrm{ReLU}(1) = 1$$

$$h_2 = \text{ReLU}(0) = 0$$
$$h_3 = \text{ReLU}(-1) = 0$$
$$y = \text{ReLU}(1 - 0 + 0) = 1$$

Case 3: (1, 0)

$$h_1 = \text{ReLU}(1) = 1$$
$$h_2 = \text{ReLU}(0) = 0$$
$$h_3 = \text{ReLU}(2) = 2$$
$$y = \text{ReLU}(1 - 0 + 2) = 3$$

Case 4: (1, 1)

$$h_1 = \text{ReLU}(2) = 2$$
$$h_2 = \text{ReLU}(1) = 1$$
$$h_3 = \text{ReLU}(1) = 1$$
$$y = \text{ReLU}(2 - 2 + 1) = 1$$

b.

Solution: Original 2-hidden-unit XOR:

- Symmetric in x_1, x_2
- Correctly separates:
  - (0,0), (1,1) → 0
  - (0,1), (1,0) → 1

With Added $h_3 = \text{ReLU}(2x_1 - x_2)$:

- Introduces asymmetry
- Gives extra weight to x_1
- Shifts decision boundary toward inputs where $x_1 = 1$

Effect:

- Output becomes biased toward x_1
- Boundary is no longer symmetric.

C.

Solution:

- The network does NOT compute XOR exactly
- Incorrect input: (1,1) → outputs 1 instead of 0.

Q8. Decision Boundary and Misclassification

You train a perceptron with decision rule:

$$y = \begin{cases} 1 & \text{if } w_1x_1 + w_2x_2 + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

with parameters w1=1, w2=−2, b=1.

Dataset:
(2,1)→1, (1,3)→0, (3,2)→1, (0,1)→0.

Tasks
a) Write the decision boundary equation and sketch it.
b) Classify each point using the perceptron rule. Which ones are misclassified?
c) Compute the perceptron loss (number of mistakes).
d) If we apply one weight update with learning rate η=0.5 on the first mistake, what are the new weights?

Solution:

Given: $w_1 = 1,\ w_2 = -2,\ b = 1$
Decision Boundary: $w_1x_1 + w_2x_2 + b = 0$

$$x_1 - 2x_2 + 1 = 0$$

b. Solution: Use: $y = \text{sign}(w \cdot x + b)$
Compute each:

(2,1): $2 - 2 + 1 = 1 \rightarrow 1$

(1,3): $1 - 6 + 1 = -4 \rightarrow 0$

(3,2): $3 - 4 + 1 = 0 \rightarrow 0$

(0,1): $0 - 2 + 1 = -1 \rightarrow 0$

Misclassification: (3,2)

c. Solution: Loss=#misclassified points = 1

d. Solution: Use perceptron update rule: $w = w + \eta \cdot t \cdot x$

$$b = b + \eta \cdot t$$

Given: $\eta = 1$
Update using misclassified point (3,2): $x = (3,2), t = 1$
New weights: $w_1 = 1 + (1)(3) = 4$

$$w_2 = -2 + (1)(2) = 0$$

New bias: $b = 1 + 1 = 2$

Part I. Short Answer

Q3. Neural Networks: Structure, Activation, and Representation

(a) Why are non-linear activation functions (like ReLU, sigmoid, tanh) necessary in neural networks?

Answer: Non-linear activation functions enable a model to capture complex patterns and relationships. Without them, the network behaves like a simple linear model.

(b) Why are deep networks better at feature learning compared to shallow models like logistic regression?

Answer: Deep networks:

- learn hierarchical features
- capture complex patterns

1) RNN Families & Use-Cases (Many-to-X)
a) Map each task to the most suitable RNN I/O pattern and explain why (1–2 lines each):
• next-word prediction

• sentiment of a sentence

• NER

• machine translation
(Choose from: one-to-many, many-to-one, many-to-many aligned, many-to-many unaligned.)

Answer:

Task	Type
Next-word prediction	One-to-many
Sentiment	Many-to-one
NER	Many-to-many (aligned)
Translation	Many-to-many (unaligned)

b) In one sentence, explain how “unrolling” over time enables BPTT and weight sharing.

Answer: Unrolling a recurrent network across time represents it as a chain of identical layers that share the same parameters, enabling backpropagation through time (BPTT) to calculate gradients over all time steps while reusing the same weights.

c) Give one advantage and one limitation of weight sharing across time in RNNs.

Answer:

• **Advantage:** Uses fewer parameters, making the model more efficient.
• **Limitation:** Lacks the ability to learn position-specific patterns or behaviors.

2) Vanishing Gradients & Remedies
a) Describe the vanishing gradient problem in RNNs and how it affects long-range dependencies.

Answer: The vanishing gradient problem happens when gradients become extremely small as they are propagated backward through many time steps in an RNN.
This occurs because repeated multiplication by small derivative values (such as those from sigmoid or tanh) causes the gradients to shrink exponentially.

Effect:
• Early time steps receive little to no learning signal
• The model struggles to capture long-term dependencies
• It mainly focuses on recent inputs while neglecting earlier context

b) List two *architectural* solutions and briefly how each helps gradient flow.

Answer: **1. LSTM (Long Short-Term Memory)**
• Uses gating mechanisms (input, forget, and output gates)
• Maintains a cell state with mostly linear connections
• Enables gradients to flow effectively, preventing vanishing
Helps retain long-term dependencies

2. GRU (Gated Recurrent Unit)
• A simplified version of LSTM with update and reset gates
• Regulates how much past information is preserved
Mitigates vanishing gradients while remaining computationally efficient

c) Give one *training* technique (not an architecture change) that can mitigate the issue and why.

Answer: **Gradient Clipping**

• Restricts the magnitude of gradients during training

• Prevents them from growing excessively large or unstable

Improves training stability and helps mitigate vanishing and exploding gradient issues

3) LSTM Gates & Cell State

a) Explain the roles of the forget, input, and output gates. For each, name its activation and purpose.

Answer: **1. Forget Gate**

• Uses a sigmoid activation

• Determines which information to remove from the cell state

• Helps eliminate irrelevant past data

• Outputs values between 0 and 1:

0. completely discard
1. completely retain

2. Input Gate

• Uses sigmoid (for gating) and tanh (for candidate values)

• Decides what new information should be added to the cell state

• Regulates how much new data is stored in memory

3. Output Gate

• Uses sigmoid (for gating) and tanh (on the cell state)

• Determines which part of the cell state becomes the hidden state

• Controls what information is passed to the next time step or layer

b) Why is the LSTM cell state often described as providing a "linear path" for gradients?

Answer: The LSTM cell state is viewed as a linear pathway because information passes through it with simple operations like addition and multiplication, without repeated nonlinear transformations.

This enables gradients to:

• Flow across many time steps with minimal change

• Avoid exponential shrinking

c) In one or two sentences, contrast "what to remember" vs. "what to expose" in LSTMs.

Answer: • **What to remember:** Determined by the forget and input gates, which control what information is retained or updated in the cell state.

• **What to expose:** Managed by the output gate, which decides which parts of the stored information are revealed as the hidden state.

4) Self-Attention

a) Define Query (Q), Key (K), and Value (V) in the context of self-attention.

Answer: In self-attention, each input token is mapped into three vectors:
• **Query (Q):** Captures what the token is trying to find or focus on
• **Key (K):** Represents what each token provides for matching
• **Value (V):** Contains the actual information carried by the token
b) Write the formula for dot-product attention.

Answer: $\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$

c) Why do we divide by $\sqrt{d_k}$?

Answer: Dividing by $\sqrt{(d_k)}$ helps reduce large dot-product values.
Without this scaling:
• Large values make the softmax output very sharp
• This can lead to unstable gradients

With scaling:
• Training becomes more stable
• The attention distribution is smoother and more balanced

5) Multi-Head Attention & Residual Connections

a) Why do Transformers use multi-head attention instead of single-head attention?

Answer: Transformers use multi-head attention rather than a single head so the model can learn different relationships simultaneously.

Each head:
• Focuses on different parts of the sequence
• Learns different types of patterns

Benefits:
• Produces richer representations
• Improves contextual understanding
b) What is the purpose of Add & Norm (Residual + LayerNorm)? Explain two benefits.

Answer: **Residual Connection (Add):** Combines the input with the sublayer output:
Output = x + Sublayer(x)

Layer Normalization (LayerNorm): Normalizes the activations across features.

Two Benefits:
• **Better Gradient Flow**
Residual connections allow gradients to pass more easily through deep networks
Helps reduce the vanishing gradient problem

• **Stable & Faster Training**
LayerNorm stabilizes activations
Leads to more consistent and quicker training
c) Describe one example of linguistic relation that different heads might capture (e.g., coreference, syntax).

Answer: A single attention head can capture relationships like coreference.
For example: "John went to the store because he needed milk."
• The model learns that "he" refers to "John"

Other relationships that attention heads can learn include:
• Syntax (such as subject–verb agreement)
• Semantic similarity
• Positional relationships

6) Encoder–Decoder with Masked Attention

a) Why does the decoder use masked self-attention? What problem does it prevent?

Answer: The decoder applies masked self-attention so that each position can only focus on earlier tokens, not future ones.**Purpose:**
• Prevents the model from "cheating" by looking at future words during training
• Maintains the left-to-right (causal) generation process

b) What is the difference between encoder self-attention and encoder–decoder cross-attention?

Answer: **Encoder Self-Attention**
• Each token attends to every other token in the input sequence
• Uses: Q = K = V = encoder inputs
• Purpose: builds contextual representations of the input

Encoder–Decoder Cross-Attention
• The decoder attends to the encoder's outputs

• Uses: Q = decoder, K and V = encoder outputs
• Purpose: helps the decoder focus on relevant parts of the input while generating the output

c) During inference (no teacher forcing), how does the model generate tokens step by step?

Answer: During inference, the model produces tokens step by step:

1. Begins with a special start token ()
2. Generates the first output token
3. Feeds that generated token back into the decoder
4. Predicts the next token
5. Repeats the process until the end token () is generated

This process is known as autoregressive generation.

Part II: Programming

Q1. Character-Level RNN Language Model ("hello" toy & beyond)
Goal: Train a tiny character-level RNN to predict the next character given previous characters.

Data (toy to start):

- Start with a small toy corpus you create (e.g., several "hello…", "help…", short words/sentences).
- Then expand to a short plain-text file of ~50–200 KB (any public-domain text of your choice).

Model:

- Embedding → RNN (Vanilla RNN or GRU or LSTM) → Linear → Softmax over characters.
- Hidden size 64–256; sequence length 50–100; batch size 64; train 5–20 epochs.

Train:

- Teacher forcing (use the true previous char as input during training).
- Cross-entropy loss; Adam optimizer.

Report:

1. Training/validation loss curves.
2. Sample 3 temperature-controlled generations (e.g., $\tau = 0.7, 1.0, 1.2$) for 200–400 chars each.

3. A 3–5 sentence reflection: what changes when you vary sequence length, hidden size, and temperature?
   (Connect to slides: embedding, sampling loop, teacher forcing, tradeoffs

   CODE:

```python
# char_rnn.py
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import random
import matplotlib.pyplot as plt

# -------------------------
# 1. Load text corpus
# -------------------------
# Replace 'sample.txt' with any public-domain text (~50-200 KB)
with open('sample.txt', 'r', encoding='utf-8') as f:
    text = f.read().lower()  # lowercase for simplicity

# Remove unwanted characters (optional)
text = ''.join(c for c in text if c.isprintable())

# Vocabulary and mappings
chars = sorted(list(set(text)))
vocab_size = len(chars)
char2idx = {ch: i for i, ch in enumerate(chars)}
idx2char = {i: ch for i, ch in enumerate(chars)}
# Encode full text
encoded_text = np.array([char2idx[c] for c in text])
print(f"Corpus length: {len(text)}, Vocab size: {vocab_size}")

"
 # 2. Create sequences
 # -------------------------
 seq_length = 50
 step = 1
 inputs, targets = [], []
 for i in range(0, len(encoded_text) - seq_length, step):
     inputs.append(encoded_text[i:i+seq_length])
     targets.append(encoded_text[i+seq_length])

 inputs = torch.tensor(np.array(inputs), dtype=torch.long)
 targets = torch.tensor(np.array(targets), dtype=torch.long)
 print("Input shape:", inputs.shape, "Target shape:", targets.shape)

 # -------------------------
 # 3. Define model
 # -------------------------
 class CharRNN(nn.Module):
     def __init__(self, vocab_size, embed_size, hidden_size):
         super().__init__()
         self.embed = nn.Embedding(vocab_size, embed_size)
         self.rnn = nn.LSTM(embed_size, hidden_size, batch_first=True)
         self.fc = nn.Linear(hidden_size, vocab_size)

     def forward(self, x, hidden=None):
         x = self.embed(x)
         out, hidden = self.rnn(x, hidden)
         out = self.fc(out[:, -1, :])  # last time step
         return out, hidden
```

```python
# 4. Hyperparameters
# -------------------------
embed_size = 128
hidden_size = 256
batch_size = 64
epochs = 10
lr = 0.002

model = CharRNN(vocab_size, embed_size, hidden_size)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=lr)
# -------------------------
# 5. Train / Validation split
# -------------------------
dataset = list(zip(inputs, targets))
random.shuffle(dataset)
split_idx = int(len(dataset) * 0.9)
train_data = dataset[:split_idx]
val_data = dataset[split_idx:]

train_losses = []
val_losses = []

# -------------------------
# 6. Training loop
# -------------------------
for epoch in range(1, epochs+1):
    # Training
    random.shuffle(train_data)
```

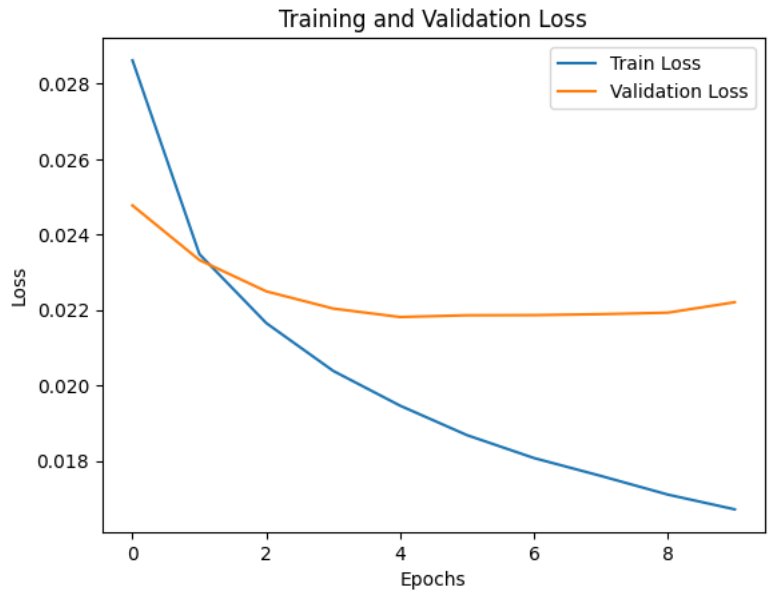
```python
# Training
random.shuffle(train_data)
total_loss = 0
for i in range(0, len(train_data), batch_size):
    batch = train_data[i:i+batch_size]
    x_batch = torch.stack([b[0] for b in batch])
    y_batch = torch.tensor([b[1] for b in batch], dtype=torch.long)

    optimizer.zero_grad()
    out, _ = model(x_batch)
    loss = criterion(out, y_batch)
    loss.backward()
    optimizer.step()
    total_loss += loss.item()
avg_train_loss = total_loss / len(train_data)
train_losses.append(avg_train_loss)

# Validation
with torch.no_grad():
    total_val_loss = 0
    for i in range(0, len(val_data), batch_size):
        batch = val_data[i:i+batch_size]
        x_batch = torch.stack([b[0] for b in batch])
        y_batch = torch.tensor([b[1] for b in batch], dtype=torch.long)
        out, _ = model(x_batch)
        loss = criterion(out, y_batch)
        total_val_loss += loss.item()
    avg_val_loss = total_val_loss / len(val_data)
    val_losses.append(avg_val_loss)
```

```python
# 7. Plot loss curves
# ------------------------
plt.plot(train_losses, label='Train Loss')
plt.plot(val_losses, label='Validation Loss')
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.title("Training and Validation Loss")
plt.show()

# ------------------------
# 8. Text generation
# ------------------------
def generate_text(model, start_seq, length=200, temperature=1.0):
    model.eval()
    generated = start_seq
    input_seq = torch.tensor([char2idx[c] for c in start_seq], dtype=torch.long).unsqueeze(0
    hidden = None
    for _ in range(length):
        out, hidden = model(input_seq, hidden)
        probs = torch.softmax(out / temperature, dim=-1).detach().numpy().flatten()
        char_idx = np.random.choice(len(probs), p=probs)
        generated += idx2char[char_idx]
        input_seq = torch.tensor([[char_idx]], dtype=torch.long)
    return generated

# Generate samples with different temperatures
start_seq = "the "
print("\n=== Temperature 0.7 ===\n", generate_text(model, start_seq, 200, temperature=0.7))
```

```python
# Generate samples with different temperatures
start_seq = "the "
print("\n=== Temperature 0.7 ===\n", generate_text(model, start_seq, 200, temperature=0.7))
print("\n=== Temperature 1.0 ===\n", generate_text(model, start_seq, 200, temperature=1.0))
print("\n=== Temperature 1.2 ===\n", generate_text(model, start_seq, 200, temperature=1.2))
```

OUTPUT

```
Corpus length: 500, Vocab size: 31
Input shape: torch.Size([450, 50]) Target shape: torch.Size([450])
Epoch 1/10 | Train Loss: 0.0545 | Val Loss: 0.0671
Epoch 2/10 | Train Loss: 0.0463 | Val Loss: 0.0621
Epoch 3/10 | Train Loss: 0.0422 | Val Loss: 0.0594
Epoch 4/10 | Train Loss: 0.0383 | Val Loss: 0.0593
Epoch 5/10 | Train Loss: 0.0352 | Val Loss: 0.0576
Epoch 6/10 | Train Loss: 0.0322 | Val Loss: 0.0566
Epoch 7/10 | Train Loss: 0.0292 | Val Loss: 0.0580
Epoch 8/10 | Train Loss: 0.0254 | Val Loss: 0.0573
Epoch 9/10 | Train Loss: 0.0229 | Val Loss: 0.0599
Epoch 10/10 | Train Loss: 0.0206 | Val Loss: 0.0597
```




```
=== Temperature 0.7 ===
 the game, and i went tone, andrepeatedin a tone of the otherworrily. "we
must begrand their hast off reallydogreat dear! no you got things!" alice
said to alice again at all a little girl abshing and

=== Temperature 1.0 ===
 the queer-feet, and book, and thought, but alice who werpents, "if i don't—
just afraniwis her _strearly," the only in a morcook. and alice has more a
long of them, and added and athand make that the pool

=== Temperature 1.2 ===
 the long, so such a greathereyalred a body thing mann! triatehise, (unfor
yourseps back toahers and on!why did you, wilish i day-of justtable."crong,
and she very know she said to creatures to a vicular t
```

Q2. Mini Transformer Encoder for Sentences

Task: Build a mini Transformer Encoder (NOT full decoder) to process a batch of sentences.

Steps:

1. Use a small dataset (e.g., 10 short sentences of your choice).
2. Tokenize and embed the text.
3. Add sinusoidal positional encoding.
4. Implement:
    - Self-attention layer
    - Multi-head attention (2 or 4 heads)
    - Feed-forward layer

- Add & Norm

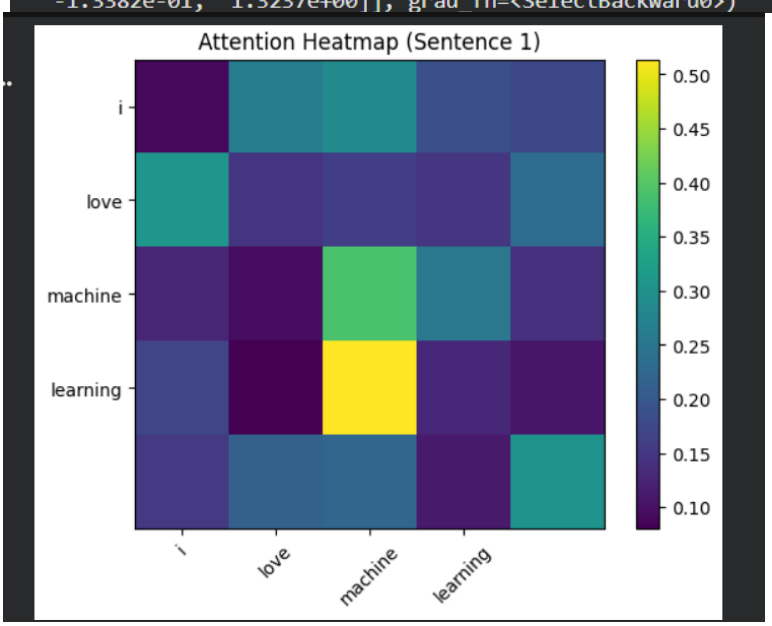
5. Show:
    - Input tokens
    - Final contextual embeddings
    - Attention heatmap between words (visual or printed)

OUTPUT:

```
Input Tokens:
 tensor([[13, 19, 20, 18,  0],
        [36,  3, 28, 22,  0],
        [ 6, 18, 15,  2,  0],
        [23, 16, 29, 15, 11],
        [ 4, 12, 22, 10,  0],
        [30, 21, 35,  7,  0],
        [13,  8, 18, 26, 33],
        [ 5, 31, 15, 14,  0],
        [25, 24, 17, 27,  0],
        [34, 15,  1, 32,  9]])
Shape: torch.Size([10, 5])

Final Contextual Embeddings Shape: torch.Size([10, 5, 32])

Sample Embedding for first sentence:
 tensor([[-1.3422e+00,  5.9908e-01, -2.4770e+00,  2.7047e-01,  2.2476e-03,
          3.3782e-01, -2.1396e+00,  1.3594e+00,  6.9069e-01,  1.6101e-01,
         -4.3949e-01,  1.7904e-01,  1.1256e+00,  1.0697e+00, -1.8618e-01,
         -9.6969e-02, -1.1997e+00,  3.2099e-01, -8.6981e-02,  1.8020e-01,
          8.7594e-02,  1.2009e-01, -1.2152e+00,  1.6954e+00, -1.1330e+00,
         -3.4049e-01, -1.0038e+00,  4.7600e-01,  9.5142e-01, -4.1138e-01,
          4.3099e-01,  2.0144e+00],
```


```
Input Tokens:
 tensor([[13, 19, 20, 18,  0],
        [36,  3, 28, 22,  0],
        [ 6, 18, 15,  2,  0],
        [23, 16, 29, 15, 11],
        [ 4, 12, 22, 10,  0],
        [30, 21, 35,  7,  0],
        [13,  8, 18, 26, 33],
        [ 5, 31, 15, 14,  0],
        [25, 24, 17, 27,  0],
        [34, 15,  1, 32,  9]])
Shape: torch.Size([10, 5])

Final Contextual Embeddings Shape: torch.Size([10, 5, 32])

Sample Embedding for first sentence:
 tensor([[-1.3422e+00,  5.9908e-01, -2.4770e+00,  2.7047e-01,  2.2476e-03,
          3.3782e-01, -2.1396e+00,  1.3594e+00,  6.9069e-01,  1.6101e-01,
         -4.3949e-01,  1.7904e-01,  1.1256e+00,  1.0697e+00, -1.8618e-01,
         -9.6969e-02, -1.1997e+00,  3.2099e-01, -8.6981e-02,  1.8020e-01,
          8.7594e-02,  1.2009e-01, -1.2152e+00,  1.6954e+00, -1.1330e+00,
         -3.4049e-01, -1.0038e+00,  4.7600e-01,  9.5142e-01, -4.1138e-01,
          4.3099e-01,  2.0144e+00],
```

```
  -9.8059e-01,  1.0378e+00],
 [ 1.2794e+00, -1.3312e+00, -9.8011e-01, -3.5084e-01,  7.0621e-01,
   1.2380e+00, -9.1327e-01,  1.7980e+00,  2.0885e-01,  1.4018e+00,
  -1.6553e+00,  1.1192e+00,  1.4827e+00,  6.0711e-01, -1.1987e+00,
   2.9377e-01, -1.0159e+00, -1.1332e+00, -3.8085e-01,  8.8784e-01,
  -8.1082e-01, -6.2329e-01, -1.3386e+00,  5.4826e-01, -2.3395e-01,
   1.6331e+00, -1.6633e-01, -5.3533e-01,  4.5640e-01, -1.6494e-01,
  -1.2248e+00,  3.9670e-01],
 [ 1.9588e-01, -1.1501e+00,  1.5506e-01, -1.1814e+00, -3.6912e-01,
   7.0207e-01,  1.6109e+00, -1.3565e-01,  8.0104e-01,  1.5895e-01,
  -1.4290e+00,  6.5506e-02,  1.1822e+00,  1.2078e+00,  8.6429e-01,
  -6.5834e-01, -1.0429e+00, -1.5594e+00, -1.5498e+00,  1.0317e+00,
  -8.5773e-01,  1.3092e+00, -1.6541e+00,  8.8757e-01,  9.4501e-01,
  -2.8118e-01, -9.4434e-01,  1.4578e+00, -6.4137e-01, -3.1037e-01,
  -1.3382e-01,  1.3237e+00]], grad_fn=<SelectBackward0>)
```



Q3. Implement Scaled Dot-Product Attention

Goal: Implement the attention function from your slides:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Requirements:

- Write a function in PyTorch or TensorFlow to compute attention.
- Test it using random Q, K, V inputs.
- Print:
  - Attention weight matrix
  - Output vectors

- Softmax stability check (before and after scaling)

OUTPUT:

```
Q:
 tensor([[[ 1.9269,  1.4873,  0.9007, -2.1055,  0.6784, -1.2345, -0.0431,
           -1.6047],
          [-0.7521,  1.6487, -0.3925, -1.4036, -0.7279, -0.5594, -0.7688,
            0.7624],
          [ 1.6423, -0.1596, -0.4974,  0.4396, -0.7581,  1.0783,  0.8008,
            1.6806],
          [ 1.2791,  1.2964,  0.6105,  1.3347, -0.2316,  0.0418, -0.2516,
            0.8599]]])

K:
 tensor([[[-1.3847, -0.8712, -0.2234,  1.7174,  0.3189, -0.4245,  0.3057,
           -0.7746],
          [-1.5576,  0.9956, -0.8798, -0.6011, -1.2742,  2.1228, -1.2347,
           -0.4879],
          [-0.9138, -0.6581,  0.0780,  0.5258, -0.4880,  1.1914, -0.8140,
           -0.7360],
          [-1.4032,  0.0360, -0.0635,  0.6756, -0.0978,  1.8446, -1.1845,
            1.3835]]])

V:
 tensor([[[ 1.4451,  0.8564,  2.2181,  0.5232,  0.3466, -0.1973, -1.0546,
            1.2780],
          [-0.1722,  0.5238,  0.0566,  0.4263,  0.5750, -0.6417, -2.2064,
           -0.7508],
          [ 0.0109, -0.3387, -1.3407, -0.5854,  0.5362,  0.5246,  1.1412,
            0.0516],
          [ 0.7440, -0.4816, -1.0495,  0.6039, -1.7223, -0.8278,  1.3347,
            0.4835]]])
```

```
==============================
Raw Attention Scores (QK^T):
[[[-5.811 -3.696 -4.362 -8.643]
  [-3.538  4.319 -1.413  1.196]
  [-3.025 -1.097 -1.438  1.458]
  [-1.579 -1.766 -1.538  0.702]]]

Scaled Scores (QK^T / sqrt(d_k)):
[[[-2.054 -1.307 -1.542 -3.056]
  [-1.251  1.527 -0.5    0.423]
  [-1.07  -0.388 -0.508  0.516]
  [-0.558 -0.624 -0.544  0.248]]]

==============================
Softmax WITHOUT scaling:
[[[0.074 0.609 0.313 0.004]
  [0.    0.955 0.003 0.042]
  [0.01  0.068 0.048 0.874]
  [0.079 0.066 0.082 0.773]]]

Softmax WITH scaling:
[[[0.194 0.41  0.324 0.071]
  [0.041 0.656 0.086 0.217]
  [0.104 0.206 0.182 0.508]
  [0.193 0.18  0.195 0.432]]]
```

```
==============================
Attention Output (Weighted Sum of V):
[[[ 0.267  0.237 -0.055  0.13   0.354 -0.191 -0.645 -0.009]
  [ 0.109  0.244 -0.216  0.381  0.063 -0.563 -1.101 -0.331]
  [ 0.495 -0.109 -0.535  0.342 -0.622 -0.477  0.322  0.233]
  [ 0.571 -0.015 -0.277  0.324 -0.468 -0.408  0.198  0.33 ]]]

==============================
Stability Check:

Max value BEFORE scaling: 4.319219589233398
Max value AFTER scaling: 1.5270748138427734

Softmax distribution WITHOUT scaling:
tensor([0.0735, 0.6092, 0.3130, 0.0043])

Softmax distribution WITH scaling:
tensor([0.1942, 0.4102, 0.3242, 0.0714])
```